

# [5회차] Basic ML 과제 보고서

주제: 클래스 불균형 상황에서의 신용카드 사기 탐지

## 1. 분석 목적

본 과제의 목적은 실제 신용카드 거래 데이터에서 극심한 클래스 불균형을 확인하고, 정상 거래를 표본추출한 뒤 SMOTE로 학습 데이터의 소수 클래스를 보강하여 사기 탐지 모델을 구축하는 것이다. 모델 평가는 단순 정확도가 아니라 Class 0(정상)과 Class 1(사기)의 Precision, Recall, F1-score 및 PR-AUC를 중심으로 수행하였다.

재현성을 위해 표본추출, 데이터 분할, SMOTE, 모델 학습의 random\_state는 모두 42로 고정하였다. 테스트셋은 최종 평가에만 사용하였으며, 모델과 threshold는 학습 데이터에서 따로 분리한 검증셋을 기준으로 선택하였다.

## 2. 데이터 로드 및 기본 탐색

사용 파일은 creditcard.csv이다. 데이터의 크기는 284,807행, 31열이며 변수는 Time, Amount, V1~V28, Class로 구성되어 있다. Class는 정상 거래가 0, 사기 거래가 1이다. 전체 데이터에서 확인된 결측치는 0개였다.

전체 클래스 분포:

| 구분          | 건수      | 비율        |
|-------------|---------|-----------|
| Class 0(정상) | 284,315 | 99.8273%  |
| Class 1(사기) | 492     | 0.1727%   |
| 합계          | 284,807 | 100.0000% |

사기 거래 비율이 약 0.17%에 불과하므로, 모든 거래를 정상으로 예측해도 정확도는 매우 높게 나타날 수 있다. 따라서 이 문제에서는 사기 거래를 얼마나 놓치지 않는지를 나타내는 Recall, Precision과 Recall의 균형을 나타내는 F1-score, 불균형 자료에서 양성 클래스의 순위 품질을 평가하는 PR-AUC가 중요하다.

## 3. 표본추출 및 전처리

과제 명세에 따라 Class 1인 사기 거래 492건은 모두 유지하였다. Class 0인 정상 거래에서는 10,000건을 random\_state=42로 무작위 추출하고, 두 데이터프레임을 결합한 뒤 행 순서를 다시 섞어 분석용 데이터프레임을 만들었다.

표본추출 후 클래스 분포:

| 구분          | 건수     | 비율        |
|-------------|--------|-----------|
| Class 0(정상) | 10,000 | 95.3107%  |
| Class 1(사기) | 492    | 4.6893%   |
| 합계          | 10,492 | 100.0000% |

전처리에서는 명세서대로 Amount 변수만 StandardScaler로 표준화하여 Amount\_Scaled 변수를 생성하고 원본 Amount 변수는 제거하였다. Time과 익명화된 PCA 변수 V1~V28은 변경하지 않았다. 이후 Class를 y로, 나머지 변수를 X로 분리하였다.

## 4. 학습/테스트 분할과 SMOTE

train\_test\_split에서 test\_size=0.2, random\_state=42, stratify=y를 사용하였다. stratify를 적용했기 때문에 학습셋과 테스트셋에 원래의 클래스 비율이 유지되었다.

| 구분   | Class 0 | Class 1 | 합계    |
|------|---------|---------|-------|
| 학습셋  | 7,999   | 394     | 8,393 |
| 테스트셋 | 2,001   | 98      | 2,099 |

SMOTE는 소수 클래스 샘플 사이의 특성 공간에서 합성 데이터를 생성하는 오버샘플링 방법이다. 원래 학습 데이터처럼 정상 거래가 압도적으로 많으면 모델이 정상 거래 쪽으로 편향되어 사기 패턴을 충분히 학습하지 못할 수 있으므로 SMOTE를 적용하였다. 데이터 누수를 방지하기 위해 테스트셋에는 SMOTE를 적용하지 않고 학습 데이터에만 적용하였다.

전체 학습셋의 SMOTE 적용 전후 클래스 수:

| 구분          | 적용 전  | 적용 후  |
|-------------|-------|-------|
| Class 0(정상) | 7,999 | 7,999 |
| Class 1(사기) | 394   | 7,999 |

SMOTE 적용 후 학습 데이터는 두 클래스가 각각 7,999건으로 균형을 이루었다.

## 5. 모델 선정 및 하이퍼파라미터 비교

비선형 관계와 변수 간 상호작용을 학습할 수 있는 트리 앙상블 모델을 중심으로 Random Forest, Extra Trees, Histogram Gradient Boosting을 비교하였다. 테스트셋을 보면서 모델을 선택하는 것을 방지하기 위해 원래 학습셋의 20%를 별도 검증셋으로 분리하였다. 이때도 SMOTE는 하위 학습셋에만 적용하고 검증셋은 원본 상태로 유지하였다.

각 후보의 threshold는 검증셋에서 Class 1 Recall이 0.80 이상인 지점 중 F1-score가 가장 높은 값으로 정하였다. 최종 모델은 검증 PR-AUC가 가장 높은 후보로 선택하였다.

검증 결과:

| 모델         | PR-AUC | threshold | Precision(1) | Recall(1) | F1(1)  |
|------------|--------|-----------|--------------|-----------|--------|
| ET_0.8     | 0.9457 | 0.3644    | 0.9589       | 0.8861    | 0.9211 |
| RF_full    | 0.9403 | 0.4457    | 0.9853       | 0.8481    | 0.9116 |
| HGB_leaf15 | 0.9397 | 0.6706    | 0.9851       | 0.8354    | 0.9041 |
| HGB_leaf31 | 0.9385 | 0.4094    | 0.9583       | 0.8734    | 0.9139 |

| 모델         | PR-AUC | threshold | Precision(1) | Recall(1) | F1(1)  |
|------------|--------|-----------|--------------|-----------|--------|
| ET_sqrt    | 0.9379 | 0.2822    | 0.9221       | 0.8987    | 0.9103 |
| RF_depth12 | 0.9363 | 0.4630    | 0.9853       | 0.8481    | 0.9116 |

가장 높은 검증 PR-AUC를 기록한 ET\_0.8을 최종 모델로 선택하였다. 이는 ExtraTreesClassifier에서 `n_estimators=450`, `max_features=0.8`, `min_samples_leaf=1`, `max_depth=None`을 사용한 모델이다. 최종 threshold는 테스트셋이 아닌 검증셋에서 결정된 0.364444를 사용하였다.

## 6. 최종 예측 및 성능 평가

선택한 Extra Trees 모델을 전체 SMOTE 학습셋으로 다시 학습하였다. `predict()`로 기본 threshold 0.5의 예측 값을 만들고, `predict_proba()`로 각 클래스의 예측 확률을 구하였다. 최종 판정은 사기 확률이 0.364444 이상이면 Class 1로 분류하는 방식으로 생성하였다.

최종 혼동행렬:

| 예측 Class 0 | 예측 Class 1 |    |
|------------|------------|----|
| 실제 Class 0 | 1,993      | 8  |
| 실제 Class 1 | 13         | 85 |

즉, 테스트셋의 사기 거래 98건 중 85건을 탐지하고 13건을 놓쳤다. 정상 거래 2,001건 중 8건을 사기로 잘못 경고하였다.

최종 테스트 지표:

| 클래스         | Precision | Recall | F1     | PR-AUC |
|-------------|-----------|--------|--------|--------|
| Class 0(정상) | 0.9935    | 0.9960 | 0.9948 | 0.9993 |
| Class 1(사기) | 0.9140    | 0.8673 | 0.8901 | 0.9276 |

목표 달성 여부:

| 클래스         | Recall $\geq 0.80$ | F1 $\geq 0.88$ | PR-AUC $\geq 0.90$ |
|-------------|--------------------|----------------|--------------------|
| Class 0(정상) | 달성                 | 달성             | 달성                 |
| Class 1(사기) | 달성                 | 달성             | 달성                 |

따라서 과제에서 제시한 Recall 0.80 이상, F1-score 0.88 이상, PR-AUC 0.90 이상의 목표를 Class 0과 Class 1 모두에서 달성하였다.

참고로 기본 threshold 0.5에서 Class 1의 Recall은 0.8673, F1-score는 0.9091이었다. 검증셋에서 선택한 0.364444를 테스트셋에 적용했을 때 사기 Recall은 동일했고 정상 거래 오탐이 4건 증가하여 Class 1 F1-score는 0.8901로 낮아졌다. 이는 테스트셋에서 0.364444와 0.5 사이의 확률을 가진 추가 사기 거래가 없었기 때문이다. 다만 threshold는 테스트셋을 본 뒤 다시 선택하지 않았으며, 사전에 정한 검증 절차를 유지한 상태에서도 모든 목표를 충족하였다.

## 7. 해석, 한계 및 추가 시도

Extra Trees는 여러 무작위 결정트리를 결합하여 복잡한 사기 거래 패턴을 학습했고, SMOTE는 학습 과정에서 사기 클래스의 정보가 충분히 반영되도록 도왔다. 최종 Class 1 PR-AUC가 0.9276이라는 것은 강한 클래스 불균형을 고려해도 사기 거래가 높은 확률 순위에 배치되었음을 의미한다. Class 1 Precision 0.9140은 모델이 사기라고 경고한 거래의 약 91.4%가 실제 사기였음을 뜻한다.

다만 본 분석에는 다음과 같은 한계가 있다.

- 1) 명세서 순서를 따르기 위해 Amount 표준화를 데이터 분할 전에 수행하였다. 실제 프로젝트에서는 scaler를 학습 데이터에만 적합하고 Pipeline으로 묶는 것이 더 엄격한 방법이다.
- 2) 정상 거래를 10,000건으로 축소하면서 실제 운영 환경의 사기 비율과 분석용 데이터의 사기 비율이 달라졌다. 따라서 실제 전체 분포에서는 Precision과 최적 threshold가 달라질 수 있다.
- 3) 한 번의 무작위 분할 결과이므로 반복 층화 교차검증을 사용하면 성능의 평균과 변동성을 더 안정적으로 평가할 수 있다.
- 4) 거래 시간을 고려하지 않은 무작위 분할이므로 시간에 따른 패턴 변화와 concept drift를 별도로 검증해야 한다.

추가 개선이 필요하다면 SMOTE의 sampling\_strategy와 k\_neighbors 조정, SMOTE 없이 class\_weight를 적용한 모델과의 비교, XGBoost/LightGBM 계열 모델, 확률 보정, 반복 교차검증, 시간 순서 기반 검증을 시도할 수 있다. 운영 threshold는 사기를 놓치는 비용과 정상 거래를 잘못 차단하는 비용을 함께 고려해 정하는 것이 바람직하다.

## 8. 최종 결론

정상 거래 10,000건과 전체 사기 거래 492건을 이용해 분석 데이터를 구성하고, Amount 표준화, 층화 8:2 분할, 학습셋 전용 SMOTE, 모델 및 하이퍼파라미터 비교, 검증셋 기반 threshold 조정을 순서대로 수행하였다. 최종 Extra Trees 모델은 독립 테스트셋에서 Class 0과 Class 1 모두 과제 목표를 충족하였다. 특히 사기 거래에 대해 Recall 0.8673, F1-score 0.8901, PR-AUC 0.9276을 기록하여 불균형 데이터에서도 의미 있는 탐지 성능을 보였다.